1. Модель прав доступа в Linux

1.1 Один пользователь, одна группа

Суть модели прав доступа в Linux видна на примере с файлами: каждый файл принадлежит одному пользователю и одной группе.
Узнать какому пользователю и какой группе принадлежит тот или иной файл можно при помощи команды ls -l.

```
┌──(kali㉿kali)-[~]
└─$ ls -l /bin/bash
-rwxr-xr-x 1 root root 1298416 Jun 22 13:05 /bin/bash

┌──(kali㉿kali)-[~]
└─$
```

В приведенном выше примере на скрине видно, что файл /bin/bash принадлежит руту и группе рута.
Модель прав доступа в Linux позволяет задать независимые права доступа для владельца, группы владельца файла и остальных пользователей.
Первое поле на скрине -rwxr-xr-x говорит нам о том, что поскольку первый символ - это минус (-), значит перед нами обычный файл.
Кроме минуса там могут быть и другие символы:

- d - директория
- l - символическая ссылка
- c - устройство символьного ввода/вывода
- b - устройство блочного ввода/вывода
- p - FIFO
- s - сокет
  Далее идут 3 тройки прав доступа, в нашем случае это rwx, r-x, r-x, что означает следующее: владельцу, то есть руту можно все, а остальным всем можно читать и исполнять файл.

1.2 whoami

Команда whoami позволяет узнать под каким пользователем мы сейчас работаем.

```
┌──(kali㉿kali)-[~]
└─$ whoami
kali
```

1.3 groups

Команды groups дает возможность увидеть к каким группам мы принадлежим.

```
┌──(kali㉿kali)-[~]
└─$ groups
kali adm dialout cdrom floppy sudo audio dip video plugdev users netdev scanner bluetooth lpadmin wireshark kab
oxer vboxsf

┌──(kali㉿kali)-[~]
```

Также можно в качестве аргумента задать имя пользователя, чтобы узнать к какой группе относится он.

```
┌──(kali㉿kali)-[~]
└─$ groups root
root : root
```

Для изменения владельца или группы файла можно использовать команды chown и chgrp соответственно.

```
┌──(root㉿kali)-[~]
└─# chown root /etc/passwd

┌──(root㉿kali)-[~]
└─# chgrp root /etc/passwd

┌──(root㉿kali)-[~]
└─#
```

Аналогично можно сменить владельца и группу файла одной командой, например chown root:root /etc/passwd.
При этом важно понимать, что chown требует прав рута, а chgrp не всегда - без прав супер пользователя он позволит изменить группу-владельца файла на ту, к которой принадлежит текущий пользователь.

Можно рекурсивно задавать права, то есть рекурсивно изменить владельца или группу у всех объектов данной директории и ниже: chown -R droppins /home/droppins.

1.4 chmod

Команда chown нужна для изменения прав доступа логики rwx, которые мы обычно видим при использовании команды ls -l.

- chmod +x - сделать файл исполняемым для пользователя, группы и всех остальных
- chmod -x - отнять все права на исполнение для всех

Для указания прав доступа конкретно владельцу, группе или всем остальным стоит использовать u, g, o соответственно.
Например chmod go-w scriptfile.sh отнимает права на запись и изменение файла у группы владельца файла и всех остальных, кроме владельца.

Можно также менять сразу все 3 права. Для этого надо использовать знак равенства.
Например chmod =rx scriptfile.txt устанавливает всем права на чтение и исполнение, но не трогает права на изменение.
Помимо этого можно сразу и указать кому именно мы переназначаем права, например

chmod u=rx scriptfile.sh устанавливает владельцу права на чтение и исполнение, но не трогает другие права и других пользователей.

1.5 Числовые режимы

То, что мы делали в предыдущем подразделе называется символическим способом указания прав доступа. Теперь рассмотрим числовые режимы. В данном случае речь пойдет о четырехзначных восьмеричных числах. Такой синтаксис называется числовым синтаксисом прав доступа. В нем каждая цифра обозначает определенное сочетание прав доступа.
Права доступа в числовой режиме выглядят следующим образом:

Режим	Число
rwx	7
rw-	6
r-x	5
r--	4
-wx	3
-w-	2
--x	1
---	0

Применение числового метода может быть полезно, когда нужно указать все разрешения для файла, как показано на рисунке ниже.

```
$ chmod 0755 scriptfile.sh
$ ls -l scriptfile.sh
-rwxr-xr-x 1 drobbins drobbins 0 Jan 9 17:44 scriptfile.sh
```

В данном случае мы назначили права -rwxr-xr-x, то есть 0755 в числовом эквиваленте.

Когда процесс создает файл, как правило он указывает, какие права надо задать для данного файла. Обычно это 0666, то есть разрешение на чтение и запись для всех. Это дает больше прав, чем надо во многих случаях.
Однако заметим, что каждый раз когда система создает новый файл, она обращается к параметру umask. Система использует этот параметр для понижения изначально задаваемые разрешения.
Набрав umask, можно увидеть какие параметры он задает.

```
File  Actions  Edit  View  Hel
┌──(kali㉿kali)-[~]
└─$ umask
0002
```

Как мы видим, наш кали очень осторожен. Настолько, что выдает всем права только на запись (как позже выяснилось, не выдает, а забирает, то есть все могут читать и исполнять, а группа и владелец также могут и изменять).
Однако настройки umask можно изменить. Попробует поставить значение 0077 вместо

0002.

```
┌──(kali㉿kali)-[~]
└─$ umask 0077

┌──(kali㉿kali)-[~]
└─$ umask
0077

┌──(kali㉿kali)-[~]
└─$
```

В отличие от обычного назначения прав доступа через chmod, здесь назначаются они от обратного, то есть перечисляется то, что надо запретить. Соответственно 0077 означает, что владелец может все, а группа и остальные не могут ничего, тк им запрещено все, то есть "---". Это значит, что в виде трех троек права по umask теперь будут выглядеть так: rwx------.

1.6 suid и sgid

В момент входа в систему запускается новый процесс в оболочки. Важно понимать, что этот новый процесс оболочки работает от имени пользователя. Следовательно при работе в оболочке bash может обратиться только к тем файлам и директориям, к которым имеет доступ пользователь. По сути мы как пользователи зависим от программа, которые выполняют операции от нашего имени. В свою очередь программы наследуют наш идентификатор пользователя, что значит, что они не могут обращаться туда, где у нас нет доступа и прав.
Например обычные пользователи не могут изменять содержимое файла /etc/passwd , потому что там отключен флаг на запись для всех кроме рута, то есть владельца.

В это де время обычным пользователям тоже надо иметь возможность как-то опосредовано менять содержимое /etc/passwd. На этот случай в модели прав доступа Linux есть 2 специальных бита, а именно suid и sgid.
Когда для запускаемой программы установлен бит suid, она будет работать от имени владельца исполняемого файла, а не от имени того, кто запустил команду.
Теперь, возвращаясь к вопросу о /etc/passwd:

```
┌──(kali㉿kali)-[~]
└─$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 118168 Apr 19 06:20 /usr/bin/passwd
```

Видим, что вместо x в триплете прав доступа стоит s. Это значит, что для этой конкретной программы установлены биты suid и права на запуск. Таким образом, запущенная программа passwd будет работать от имени рута, а не от имени того, кто ее запустил. А поскольку программа passwd работает от имени рута, то и редактировать /etc/passwd она может.

sgid работает примерно также, как и suid. Она позволяет программе наследовать права доступа группы, а не текущего пользователя.

```
биты suid и sgid занимают те же поля в триплете прав доступа, что и право на
исполнение, .поэтому если бит x тоже задан, то би будет показан как s без
шифта, а если не задан x, то отобразится S, то есть с шифтом.
suid и sgid могут быть удобны во многих ситуациях, однако неправильное
использование этих битов может привести к появлению уязвимостей в системе.
Чем меньше suid программ, тем лучше. Команда passwd – одна из немногих,
которая должна быть suid.
```

- chmod u+s - установка битов suid и sgid
- chmod u-s - удаление битов suid и sgid

1.7 Права и директории

Выше мы проговорили о правах доступа относительно обычных файлов. Теперь следует посмотреть как это работает с директориями.
В целом директории используют те же флаги прав доступа, но их интерпретация несколько отличается.

- чтение - можно просматривать список содержимого директории
- запись - можно создавать файлы в директории
- исполнение - можно войти в директорию и обращаться ко всем поддиректориям внутри. Без флага исполнения фактически не будет прав доступа к объектам внутри директории.

Если для директории установлен флаг sgid. то все объекты файловой системы, которые создаются внутри, наследуют группу директории. Это бывает полезно, когда надо создать дерево директорий и все они должны принадлежать одной группе.
Например:

```
# mkdir /home/groupspace
# chgrp mygroup /home/groupspace
# chmod g+s /home/groupspace
```

Теперь все пользователи группы mygroup имеют право на создание файлов внутри директории. Также им будет автоматом задана принадлежность к группе mygroup.

Касательно удаления и переименования, зачастую по умолчанию кто угодно может переименовать нашу директорию или даже удалить ее, если у них есть права на запись здесь.
Однако интереснее дело обстоит с директориями, которыми пользуется большое количество пользователей, например /tmp, /var/tmp. В этом случае это может вызвать много проблем, потому что кто угодно может удалять и переименовывать чьи угодно файлы, даже в случае, если они ему не принадлежат.
На такой случай в Linux есть бит sticky. Когда для /tmp установлен sticky бит (например

при помощи команды chmod +t), единственные, кто может удалить или переименовать файлы в /tmp - это либо владельцы, либо рут.

Первый знак в численном синтаксисе отвечает за задание битов sticky, suid и sgid:

suid	sgid	sticky	режим
+	+	+	7
+	+	-	6
+	-	+	5
+	-	-	4
-	+	+	3
-	+	-	2
-	-	+	1
-	-	-	0

Для установки прав доступа на директорию, которая будет использоваться рабочей группой можно задать права доступа, используя 4-значный режим, следующим образом:
chmod 1775 /home/groupfiles
То есть мы устанавливаем, что включен бит sticky, у владельца и группы все права, а у остальных пользователей права только на чтение и исполнение.

2. Управление аккаунтами в Linux

2.1 /etc/passwd

В файле /etc/passwd определены все пользователи, существующие в системе.
При помощи команды less /etc/passwd можно посмотреть этот файл.

```
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
```

Выше представлен пример из моего /etc/passwd.
Первое поле - это имя пользователя, второе - "x" (раньше здесь был шифрованный пароль для аутентификации, но теперь он хранится в другом файле), третье - это числовой идентификатор, четвертое - нужно для ассоциации конкретного пользователя с какой-либо группой, пятое - текстовое описание аккаунта либо имя пользователя, шестое - определение домашнего каталога пользователей, седьмое - стартовая оболочка пользователя.
В системе также есть взаимосвязанный с /etc/passwd файл, а именно /etc/shadow. В отличие от /etc/passwd он доступен на чтение только руту и содержит шифрованную

информацию о паролях. Это выглядит примерно так:
drobbins:$1$12345678901234567890123456789 01:11664:0:-1:-1:-1:-1:0.

Каждая строка здесь определяет информацию о пароле конкретного аккаунта, а поля в ней разделены знаком ":".

- поле 1 - определение конкретного пользовательского аккаунта
- поле 2 - зашифрованный пароль
- поле 3 - количество дней с 01.01.1970 до момента, когда пароль был изменен
- поле 4 — количество дней до того, как будет разрешено сменить пароль («0» — «менять в любое время»)
- поле 5 — количество дней до того, как система заставит пользователя сменить пароль ("-1" — «никогда»)
- поле 6 — количество дней до истечения срока действия пароля, когда пользователь получит предупреждение об этом ("-1" — «не предупреждать»)
- поле 7 — количество дней после истечения срока действия пароля, по прошествии которых аккаунт будет автоматически отключен системой ("-1" — «не отключать»)
- поле 8 — количество дней, прошедшее с момента отключения этого аккаунта ("-1" — «этот аккаунт включен»)
- поле 9 — зарезервировано для будущего использования

2.2 /etc/group

Файл /etc/group определяет группы. Например :

```
proxy:x:13:
kmem:x:15:
dialout:x:20:kali
fax:x:21:
voice:x:22:
cdrom:x:24:kali
floppy:x:25:kali
tape:x:26:
sudo:x:27:kali
audio:x:29:pulse,kali
dip:x:30:kali
www-data:x:33:
backup:x:34:
operator:x:37:
list:x:38:
irc:x:39:
src:x:40:
shadow:x:42:
utmp:x:43:
video:x:44:kali
sasl:x:45:
plugdev:x:46:kali
/etc/group
```

- поле 1 - имя группы
- поле 2 - остаточный пароль. Это поле сейчас зарезервировано x
- поле 3 - числовой идентификатор группы

- поле 4 - определение всех членов группы

2.3 Ручное создание пользователей и групп

Для изменения системных файлов следует использовать команды vipw и vigr, тк они имеют повышенные меры предосторожности, оберегая файлы от участи быть испорченными.

Запустив команду vigr -p от имени рута для редактирования файла /etc/passwd, добавим туда строчку testuser:x:3000:3000:LPI tutorial test user:/home/testuser:/bin/false.

```
root@kali: ~
File  Actions  Edit  View  Help
  GNU nano 8.4                    /etc/passwd.edit *
dnsmasq:x:999:65534:dnsmasq:/var/lib/misc:/usr/sbin/nologin
avahi:x:105:108:Avahi mDNS daemon:/run/avahi-daemon:/usr/sbin/nologin
nm-openvpn:x:106:109:NetworkManager OpenVPN:/var/lib/openvpn/chroot:/usr/sbin/nologin
speech-dispatcher:x:107:29:Speech Dispatcher:/run/speech-dispatcher:/bin/false
usbmux:x:108:46:usbmux daemon:/var/lib/usbmux:/usr/sbin/nologin
nm-openconnect:x:109:110:NetworkManager OpenConnect plugin:/var/lib/NetworkManager:/usr/sbin/nologin
pulse:x:110:111:PulseAudio daemon:/run/pulse:/usr/sbin/nologin
lightdm:x:111:114:Light Display Manager:/var/lib/lightdm:/bin/false
statd:x:112:65534::/var/lib/nfs:/usr/sbin/nologin
saned:x:113:115::/var/lib/saned:/usr/sbin/nologin
polkitd:x:989:989:User for polkitd:/:/usr/sbin/nologin
rtkit:x:114:116:RealtimeKit:/proc:/usr/sbin/nologin
colord:x:115:117:colord colour management daemon:/var/lib/colord:/usr/sbin/nologin
mysql:x:116:119:MariaDB Server:/nonexistent:/bin/false
stunnel4:x:988:988:stunnel service system account:/var/run/stunnel4:/usr/sbin/nologin
geoclue:x:117:120::/var/lib/geoclue:/usr/sbin/nologin
Debian-snmp:x:118:121::/var/lib/snmp:/bin/false
sslh:x:119:122::/nonexistent:/usr/sbin/nologin
cups-pk-helper:x:120:125:user for cups-pk-helper service:/nonexistent:/usr/sbin/nologin
redsocks:x:121:126::/var/run/redsocks:/usr/sbin/nologin
_gophish:x:122:128::/var/lib/gophish:/usr/sbin/nologin
iodine:x:123:65534::/run/iodine:/usr/sbin/nologin
miredo:x:124:65534::/var/run/miredo:/usr/sbin/nologin
redis:x:125:129::/var/lib/redis:/usr/sbin/nologin
postgres:x:126:130:PostgreSQL administrator:/var/lib/postgresql:/bin/bash
mosquitto:x:127:131::/var/lib/mosquitto:/usr/sbin/nologin
inetsim:x:128:132::/var/lib/inetsim:/usr/sbin/nologin
_gvm:x:129:134::/var/lib/openvas:/usr/sbin/nologin
kali:x:1000:1000::/home/kali:/bin/bash
sshuser:x:1001:1001:userr,1,,:/home/sshuser:/bin/bash
testuser

^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```

```
inetsim:x:128:132::/var/lib/inetsim:/usr/sbin/nologin
_gvm:x:129:134::/var/lib/openvas:/usr/sbin/nologin
kali:x:1000:1000::/home/kali:/bin/bash
sshuser:x:1001:1001:userr,1,,:/home/sshuser:/bin/bash
testuser:x:3000:3000:LPI tutorial test user:/home/testuser:/bin/false

^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C
```

Теперь посмотрим этот файл через less чтобы убедиться в том, что мы все добавили:

```
_gvm:x:129:134::/var/lib/openvas:/usr/sbin/nologin
kali:x:1000:1000::/home/kali:/bin/bash
sshuser:x:1001:1001:userr,1,,:/home/sshuser:/bin/bash
testuser:x:3000:3000:LPI tutorial test user:/home/testuser:/bin/false
(END)
```

Как видно, теперь действительно там есть еще один пользователь testuser с идентификатором 3000. Он определен в группу с таким же идентификатором.

Теперь добавим этого же пользователя в файл /etc/shadow. Для этого используем команду vigr -s. После этого откроется редактор nano, где мы и будем изменять файл.

```
File  Actions  Edit  View  Help
  GNU nano 8.4                    /etc/gshadow.edit
root:*::
daemon:*::
bin:*::
sys:*::
adm:*::kali
tty:*::
disk:*::
lp:*::
mail:*::
news:*::
uucp:*::
man:*::
proxy:*::
kmem:*::
dialout:*::kali
fax:*::
voice:*::
cdrom:*::kali
floppy:*::kali
tape:*::
sudo:*::kali
audio:*::pulse,kali
dip:*::kali
www-data:*::
backup:*::
operator:*::
list:*::
irc:*::
src:*::
shadow:*::
utmp:*::
                              [ Read 89 lines ]
^G Help      ^O Write Out   ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```

Запишем туда данные нового пользователя, взяв за шаблон строку обычного пользователя (kali).

```
lpadmin:!::kali
redsocks:!::
kismet:!::
_gophish:!::
sambashare:!::
winbindd_priv:!::
redis:!::_gvm
postgres:!::
mosquitto:!::
inetsim:!::
wireshark:!::kali
_gvm:!::
kali:!::
kaboxer:!::kali
vboxsf:!::kali
sshuser:!::
testuser:$y$j9T$bg389SaiV3HF5QvdxHBw0/$paE0h9raQ5jUZtIDFJvpOS3NhLOoUh8S58mXoNoguo7:20254:0:99999:7

^G Help      ^O Write Out   ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```

2.4 Установка пароля

При помощи команды passwd от имени рута мы можем менять пароль пользователя. Установим пароль для testuser.

```
┌──(root㉿kali)-[~]
└─# passwd testuser
New password:
Retype new password:
passwd: password updated successfully

┌──(root㉿kali)-[~]
```

pswd-kali-testuser

Далее приступим к редактированию /etc/group. Для этого используем vigr. Откроется nano с файлом /etc/group.

```
File  Actions  Edit  View  Help
  GNU nano 8.4                    /etc/group.edit
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:kali
tty:x:5:
disk:x:6:
lp:x:7:
mail:x:8:
news:x:9:
uucp:x:10:
man:x:12:
proxy:x:13:
kmem:x:15:
dialout:x:20:kali
fax:x:21:
voice:x:22:
cdrom:x:24:kali
floppy:x:25:kali
tape:x:26:
sudo:x:27:kali
audio:x:29:pulse,kali
dip:x:30:kali
www-data:x:33:
backup:x:34:
operator:x:37:
list:x:38:
irc:x:39:
src:x:40:
shadow:x:42:
utmp:x:43:
                                  [ Read 89 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```

Добавим строку для нового пользователя.

```
wireshark:x:133:kali
_gvm:x:134:
kali:x:1000:
kaboxer:x:135:kali
vboxsf:x:136:kali
sshuser:x:1001:
testuser:x:3000:

^G Help      ^O Write Out   ^F W
^X Exit      ^R Read File   ^\ R
```

2.5 Создание домашней директории

Для создания домашней директории нового пользователя введем следующие команды:

```
┌──(root㉿kali)-[~]
└─# cd /home

┌──(root㉿kali)-[/home]
└─# mkdir testuser

┌──(root㉿kali)-[/home]
└─# chown testuser:testuser testuser

┌──(root㉿kali)-[/home]
└─# chmod o-rwx testuser

┌──(root㉿kali)-[/home]
└─#
```

Таким образом, мы создали домашнюю директорию, прописали группы и пользователя для нее, а также права.
поскольку это тестовый аккаунт, то на нем оболочка написана /bin/false. Это значит, что если мы хотим использовать этого пользователя, то придется через команду vipw изменить стартовую оболочку на /bin/bash/

3. Утилиты администрирования учетных записей

- newgrp - по умолчанию, любой файл, созданный пользователем, сразу присваивается к группе, в которой состоит пользователь. Эта группа определена в /etc/passwd. Если же пользователь принадлежит к другим группам, то можно набрать newgrp thisgroup чтобы стать членом группы thisgroup. Далее любые новые созданные файлы унаследуют членство в thisgroup.
- chage - просмотр и извлечение настроек срока действия паролей, который сохранены в /etc/shadow
- gpasswd - основная утилита для управления группами
- groupadd/groupdel/groupmod - добавление/удаление/изменение групп в /etc/group
- useradd/userdel/usermod - добавление/удаление/изменение пользователей в /etc/passwd
- pwdconv|grpconv - преобразование passwd и group файлов старого образца в новые shadow passwords

4. Настройка пользовательского окружения

4.1 fortune

В нашем случае на кали не было этой утилиты, поэтому ставим ее через apt-get install fortune.

```
┌──(root㉿kali)-[~]
└─# apt-get install fortune
Reading package lists ... Done
Building dependency tree ... Done
Reading state information ... Done
Note, selecting 'fortune-mod' instead of 'fortune'
The following additional packages will be installed:
  fortunes-min librecode3
Suggested packages:
  fortunes
The following NEW packages will be installed:
  fortune-mod fortunes-min librecode3
0 upgraded, 3 newly installed, 0 to remove and 0 not upgraded.
Need to get 330 kB of archives.
After this operation, 1,648 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://kali.download/kali kali-rolling/main amd64 librecode3 amd64 3.7.15-1 [235 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 fortunes-min all 1:1.99.1-9 [56.5 kB]
Get:2 http://mirror.krfoss.org/kali kali-rolling/main amd64 fortune-mod amd64 1:1.99.1-9 [38.7 kB]
Fetched 330 kB in 22s (14.7 kB/s)
Selecting previously unselected package librecode3:amd64.
(Reading database ... 419360 files and directories currently installed.)
Preparing to unpack .../librecode3_3.7.15-1_amd64.deb ...
Unpacking librecode3:amd64 (3.7.15-1) ...
Selecting previously unselected package fortune-mod.
Preparing to unpack .../fortune-mod_1%3a1.99.1-9_amd64.deb ...
Unpacking fortune-mod (1:1.99.1-9) ...
Selecting previously unselected package fortunes-min.
Preparing to unpack .../fortunes-min_1%3a1.99.1-9_all.deb ...
```

Эта утилита не какая-то супер крутая хакерская штучка, скорее из разряда мотивационных обоев с подписью "не сдавайся". При использовании fortune появится дружелюбное сообщение, например:

```
┌──(kali㉿kali)-[~]
└─$ fortune
Clothes make the man.  Naked people have little or no influence on society.
                -- Mark Twain

┌──(kali㉿kali)-[~]
```

Мелочь, а приятно.

Попробуем настроить все так, чтобы fortune запускалась каждый раз когда мы входим. Для этого через nano отредактируем файл .bash_profile (он лежит в домашней директории). Если такого файла нет, то можно просто создать его.
У нас такого файла не было, поэтому мы его создали через touch ~/.bash_profile. Далее мы открыли его в nano и написали одну строчку "fortune". Теперь, по идее после перезагрузки все должно начать работать как задумано.
Короче скрин сделать для описания этого не удалось, но в целом идея выглядит так: пока система запускается, а графический интерфейс еще не включился, то есть до запуска менеджера дисплея, мы видим приветствие по типу такого: mycroft.flatmonk.org
login: chouser
Password:
Freedom from incrustations of grime is contiguous to rectitude.

4.2 Оболочка входа

При запуске bash проходит файл .bash_profile так, будто набирает каждую строчку в консоли. Это называется интерпретацией файла (file sourcing).

Есть 2 способа запуска bash.

1. первый вход в систему. bash запустится с именем процесса -bash.

```
$ ps u
USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND
chouser 404 0.0 0.0 2508 156 tty2 S 2001 0:00 -bash
```

2. использование опции --login.

После авторизации у нас появляется возможность запустить много копий нашей оболочки. У тех оболочек, что запущены без опции --login или не имеют черточки перед именем процесса, будут работать иначе, чем те, что при запуске с авторизацией. Они называются интерактивными оболочками. Если bash запущена в интерактивном режиме, без авторизации, то она игнорирует /etc/profile и ~/.bash_profile. Вместо этого она обработает ~/.bashrc.

interactive	login	profile	rc
yes	yes	source	ignore
yes	no	ignore	source
no	yes	source	ignore
no	no	ignore	ignore

Но также bash может и обрабатывать ~/.bashrc не будучи запущенным интерактивно. Например это может произойти при использовании команд rsh или scp.
Для проверки факта интерактивности текущей сессии можно использовать переменную PS1:

```
┌──(kali㉿kali)-[~]
└─$ if [ -n "$PS1" ]; then fortune;  fi
Patch griefs with proverbs.
                -- William Shakespeare, "Much Ado About Nothing"
```

/etc/profile отвечает за правила для новых пользователей, которые входят первый раз. /etc/skel - при использовании команды useradd для создания нового пользователя, все файлы из /etc/skel копируются в домашний каталог нового пользователя. Это значит, что можно например положить заранее туда .bashrc или .bash_profile.

Переменные в bash могут быть помечены так, чтобы они устанавливались во всех новых оболочках (вновь запускаемых оболочках). Это значит, что они обозначены как внешние (export).

Используя команду export можно посмотреть список таких внешних переменных.

```
┌──(kali㉿kali)-[~]
└─$ export
declare -x COLORFGBG="15;0"
declare -x COLORTERM="truecolor"
declare -x COMMAND_NOT_FOUND_INSTALL_PROMPT="1"
declare -x DBUS_SESSION_BUS_ADDRESS="unix:path=/run/user/1000/bus"
declare -x DESKTOP_SESSION="lightdm-xsession"
declare -x DISPLAY=":0.0"
declare -x DOTNET_CLI_TELEMETRY_OPTOUT="1"
declare -x GDMSESSION="lightdm-xsession"
declare -x HOME="/home/kali"
declare -x LANG="en_US.UTF-8"
declare -x LANGUAGE=""
declare -x LESS_TERMCAP_mb=$'\E[1;31m'
declare -x LESS_TERMCAP_md=$'\E[1;36m'
declare -x LESS_TERMCAP_me=$'\E[0m'
declare -x LESS_TERMCAP_se=$'\E[0m'
declare -x LESS_TERMCAP_so=$'\E[01;33m'
declare -x LESS_TERMCAP_ue=$'\E[0m'
declare -x LESS_TERMCAP_us=$'\E[1;32m'
declare -x LOGNAME="kali"
declare -x LS_COLORS="rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;01
01:mi=00:su=37;41:sg=30;43:ca=00:tw=30;42:ow=34;42:st=37;44:ex=01;32:*.7z=01;31:*.ace=01;
1;31:*.arc=01;31:*.arj=01;31:*.bz=01;31:*.bz2=01;31:*.cab=01;31:*.cpio=01;31:*.crate=01;3
1;31:*.dwm=01;31:*.dz=01;31:*.ear=01;31:*.egg=01;31:*.esd=01;31:*.gz=01;31:*.jar=01;31:*.
*.lz=01;31:*.lz4=01;31:*.lzh=01;31:*.lzma=01;31:*.lzo=01;31:*.pyz=01;31:*.rar=01;31:*.rpm
```

4.3 Экспортирование переменных

В случае, когда переменная не отмечена на экспорт, ее значение не будет задано для новых запускаемых оболочек. Однако можно отметить переменную для экспорта, передав ее встроенной команду export.

```
┌──(kali㉿kali)-[~]
└─$ FOO=foo

┌──(kali㉿kali)-[~]
└─$ BAR=bar

┌──(kali㉿kali)-[~]
└─$ export BAR

┌──(kali㉿kali)-[~]
└─$ echo $FOO $BAR
foo bar

┌──(kali㉿kali)-[~]
└─$ bash
┌──(kali㉿kali)-[~]
└─$ echo $FOO $BAR
bar

┌──(kali㉿kali)-[~]
└─$
```

Были заданы обе переменные FOO и BAR, но экспортирована только BAR. Когда был запущен новый bash, он потерял FOO. При этом если выйти из нового bash, то мы

увидим, что значения переменных не изменились.

```
┌──(kali㉿kali)-[~]
└─$ exit
exit

┌──(kali㉿kali)-[~]
└─$ echo $FOO $BAR
foo bar

┌──(kali㉿kali)-[~]
└─$
```

Таким образом, переменные могут быть указаны в ~/.bashprofile или /etc/profile и помечены для экспорта, для того, чтобы в дальнейшем не было необходимости указывать их снова.
set - настройка опций, которые не могут быть экспортированы.

- set -x - вывод на экран каждой команды, которые собирается выполнить bash. Это полезно для понимания непредвиденного поведения команд при использовании кавычек или похожих странностей
- set +x - выключение опции set -x
  Также set можно использовать для задания значений переменных. Однако при этом указание самой команды не будет обязательным. Например set FOO=foo делает тоже, что и FOO=foo. Сброс значения переменной делается следующим образом:

```
┌──(kali㉿kali)-[~]
└─$ FOO=bar

┌──(kali㉿kali)-[~]
└─$ echo $FOO
bar

┌──(kali㉿kali)-[~]
└─$ unset FOO

┌──(kali㉿kali)-[~]
└─$ echo $FOO


┌──(kali㉿kali)-[~]
└─$
```

  Надо понимать, что это не тоже самое, что установка переменной пустым значением.
  Наглядная разница между пустым значением и обнулением переменной показана ниже.

```
┌──(kali㉿kali)-[~]
└─$ FOO=bar

┌──(kali㉿kali)-[~]
└─$ set | grep ^FOO
FOO=bar

┌──(kali㉿kali)-[~]
└─$ FOO=

┌──(kali㉿kali)-[~]
└─$ set | grep ^FOO
FOO=

┌──(kali㉿kali)-[~]
└─$ unset FOO

┌──(kali㉿kali)-[~]
└─$ set | grep ^FOO

┌──(kali㉿kali)-[~]
└─$
```

Если использовать set без параметров, то в целом он выводит примерно тоже, что и export, но при этом отображает все переменные, а не только те, которые обозначены как внешние.

Зачастую можно изменить поведение программ установкой переменных окружения. Запускаемые программы из текущей сессии командной строки видят только те переменные окружения, которые помечены на экспорт.
К примеру команды man проверяет переменную PAGER для выяснения какую программу надо использовать для постраничного просмотра текста.
Внесем корректировки в переменную:

```
┌──(kali㉿kali)-[~]
└─$ PAGER=less

┌──(kali㉿kali)-[~]
└─$ export PAGER

┌──(kali㉿kali)-[~]
└─$ man man
```

```
File  Actions  Edit  View  Help
MAN(1)                          Manual pager utils                          MAN(1)

NAME
       man - an interface to the system reference manuals

SYNOPSIS
       man [man options] [[section] page ...] ...
       man -k [apropos options] regexp ...
       man -K [man options] [section] term ...
       man -f [whatis options] page ...
       man -l [man options] file ...
       man -w|-W [man options] page ...

DESCRIPTION
       man  is  the system's manual pager.  Each page argument given to man is normally the name of a program,
       utility or function.  The manual page associated with each of these arguments is then  found  and  dis-
       played.   A  section, if provided, will direct man to look only in that section of the manual.  The de-
       fault action is to search in all of the available sections  following  a  pre-defined  order  (see  DE-
       FAULTS), and to show only the first page found, even if page exists in several sections.

       The table below shows the section numbers of the manual followed by the types of pages they contain.

       1   Executable programs or shell commands
       2   System calls (functions provided by the kernel)
       3   Library calls (functions within program libraries)
       4   Special files (usually found in /dev)
       5   File formats and conventions, e.g. /etc/passwd
       6   Games
       7   Miscellaneous (including macro packages and conventions), e.g. man(7), groff(7), man-pages(7)
       8   System administration commands (usually only for root)
       9   Kernel routines [Non standard]

       A manual page consists of several sections.

 Manual page man(1) line 1 (press h for help or q to quit)
```

А теперь еще раз изменим, но уже вместо less поставим cat.

```
┌──(kali㉿kali)-[~]
└─$ PAGER=cat

┌──(kali㉿kali)-[~]
└─$ export PAGER

┌──(kali㉿kali)-[~]
└─$ man man
```

```
File  Actions  Edit  View  Help
     man ls
         Display the manual page for the item (program) ls.

     man man.7
         Display  the manual page for macro package man from section 7.  (This is an alternative spelling of
         "man 7 man".)

     man 'man(7)'
         Display the manual page for macro package  man  from  section  7.  (This  is  another  alternative
         spelling  of  "man  7 man".  It may be more convenient when copying and pasting cross-references to
         manual pages.  Note that the parentheses must normally be quoted to protect them from the shell.)

     man -a intro
         Display, in succession, all of the available intro manual pages contained within the manual.  It is
         possible to quit between successive displays or skip any of them.

     man -t bash | lpr -Pps
         Format the manual page for bash into the default troff or groff format and pipe it to  the  printer
         named  ps.  The  default  output  for groff is usually PostScript.  man --help should advise as to
         which processor is bound to the -t option.

     man -l -Tdvi ./foo.1x.gz > ./foo.1x.dvi
         This command will decompress and format the nroff source manual page ./foo.1x.gz into a device  in-
         dependent  (dvi) file.  The redirection is necessary as the -T flag causes output to be directed to
         stdout with no pager.  The output could be viewed with a program such as xdvi or further  processed
         into PostScript using a program such as dvips.

     man -k printf
         Search  the  short descriptions and manual page names for the keyword printf as regular expression.
         Print out any matches.  Equivalent to apropos printf.

     man -f smail
         Lookup the manual pages referenced by smail and print out the  short  descriptions  of  any  found.
         Equivalent to whatis smail.
```

И, как мы видим, теперь текст выведен уже через cat.

Если забыть установить PAGER обратно в less, то некоторые программы так и продолжат выводить текст без остановки. В таком случае можно воспользоваться командой env.

```
$ PAGER=less
$ env PAGER=cat man man
$ echo $PAGER
less
```

5. Файловые системы, разделы и блочные устройства

5.1 Введение в блочные устройства

Блочные устройства: первый диск IDE в системе, который будет называться /dev/hda (/dev/sda).
Блочные устройства представляют собой абстрактный интерфейс к диску.
Пользовательские программы используют блочные устройства для взаимодействия с диском, не беспокоясь о том, что у нас за диски: IDE, SCSI или какие-то еще.
Программы легко переадресовывают место на диске как последовательность блоков по 512 байт с произвольным доступом.

В Linux файловые системы создаются при помощи команды mkfs, где в качестве аргумента указывается конкретное блочное устройство.

Однако, хотя и возможно использовать блочные устройства, представляющие весь диск целиком, такие как /dev/hda или /dev/sda, для единственной ФС, это редко применяется на практике. Вместо этого дисковые блочные устройства разделяются на более удобные блочные устройства меньшего размера, называемые разделами. Разделы создаются с помощью средства под названием fdisk, которое используется для создания и редактирования таблиц разделов, расположенных на каждом диске. Таблица разделов определяет, как именно разбито пространство на целом диске.

5.2 Введение в fdisk

При помощи команды fdisk можно посмотреть на таблицу разделов конкретного блочного устройства.
рекомендуется не использовать cfdisk в качестве аналога, тк он иногда неправильно расчитывает геометрию диска.

```
не стоит сохранять и вносить изменения в дисковую таблицу разделов, если
один из них содержит файловую систему, используемую здесь и сейчас. Это
может привести к потере данных на диске.
```

Используя опцию p можно посмотреть текущую таблицу разделов.

```
Command (m for help): p

Disk /dev/sda: 80.09 GiB, 86000000000 bytes, 167968750 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x3a69c69b

Device     Boot Start       End   Sectors  Size Id Type
/dev/sda1  *     2048 167968749 167966702 80.1G 83 Linux

Command (m for help):
```

Как видно, в данном случае размечена одна файловая система.
В начале эры ПК, программы разметки позволяли делать максимум четыре раздела (называемых первичными). Этого было слишком мало, и для обхода этого ограничения был придуман расширенный раздел. Расширенный раздел очень похож на первичный, и засчитываются в лимит для 4-х первичных разделов. Но, расширенный раздел может содержать любое количество т. н. логических разделов внутри себя, эффективно обходя ограничение на четыре раздела.
У каждого раздела есть ID, то есть тип раздела. Каждый раз при создании нового раздела надо убедиться, что тип верно указан.

- 83 - разделы ФС Linux
- 82 - разделы подкачки
  Используя опцию t можно установить значение типа.

5.4 Создание файловых систем

Основные типы ФС:

ext2 является проверенной годами файловой системой Linux, но она не обладает средствами журналирования метаданных, что означает, что время на проверку файловой системы во время запуска может быть довольно большим. В настоящее время существует широкий выбор журналируемых файловых систем, которые могут быть проверены на целостность очень быстро, и потому предпочтительны, нежели их не журналируемые аналоги. Журналируемая ФС позволяет избежать долгих задержек при старте системы, когда целостность вашей ФС нарушена.

ext3 – журналируемая версия файловой системы ext2, которая обеспечивает журналирование метаданных для быстрого восстановления, а также другие режимы журналирования, такие как полное журналирование всех данных и упорядоченное журналирование. ext3 – очень хорошая и надежная ФС. Она предлагает достойную производительность в большинстве случаев. Поскольку она мало использует «деревья» в своем внутреннем устройстве, она плохо масштабируется, это означает, что этот тип ФС не лучший выбор для очень больших файловых систем, или в условиях, когда вы будете обрабатывать большие файлы или большое количество файлов в одном каталоге. Но при использовании её в условиях, под которые она проектировалась, ext3 прекрасная файловая система.

Одна из приятных особенностей ext3 – это то, что существующие системы ext2 могу быть обновлены «на месте» до ext3 довольно просто. Это позволяет плавно обновлять существующие системы Linux, которые уже используют ext2.

ReiserFS – это файловая система, основанная на B-дереве, которая имеет очень хорошую производительность и значительно превосходит ext2 и ext3 при работе с небольшими файлами (файлы менее 4 КБ), часто в 10-15 раз. А также ReiserFS отлично масштабируется и имеет журналирование метаданных. Начиная с ядра версии 2.4.18 и выше, ReiserFS является стабильной и рекомендуется, как в качестве ФС общего назначения, так и в крайних случаях, таких как создание больших файловых систем, использование для множества маленьких файлов, для огромных файлов, а также для каталогов с десятками тысяч файлов. Мы рекомендуем ФС ReiserFS для использования по умолчанию для всех не загрузочных разделов.

XFS – это файловая система с журналированием метаданных. Она обладает конкретным набором возможностей и оптимизирована для масштабирования. Мы рекомендуем использовать эту файловую систему исключительно на Linux системах с высококлассными SCSI и/или Fibre Channel накопителями и источниками

бесперебойного питания. Поскольку XFS агрессивно кэширует данные в ОЗУ, неподходяще спроектированная программа (т. е. та, которая не принимает должной предосторожности при записи на диск) может потерять порцию данных, если система неожиданно даст сбой.

JFS является созданной в IBM высокопроизводительной журналируемой файловой системой. В последнее время она стала предустановленной, и мы бы хотели накопить больший опыт её использования, прежде чем выявлять сильные и слабые стороны этой файловой системы.

ext4 — журналируемая файловая система, используемая преимущественно в операционных системах с ядром Linux. Создана на базе ext3, но предлагает улучшения в производительности, масштабируемости и надёжности.

btrfs — файловая система для Linux, основанная на структурах B-деревьев и работающая по принципу «копирование при записи» (copy-on-write). Опубликована корпорацией Oracle в 2007 году под лицензией GNU General Public License.

mkswap - команда для инициализации раздела подкачки
swapon - активация раздела подкачки (раздел подкачки не монтируется)

Можно использовать команду cat /proc/swaps чтобы узнать какие разделы подкачки сейчас используются.
mke2fs - создание ФС ext2
mke2fs -j - создание ФС ext3
mkreiserfs - создание ФС ReiserFS
mkfs.xfs - создание ФС XFS
mkfs.jfs - создание ФС JFS

5.5 Монтирование файловых систем

mount /dev/sda2 /mnt - команда для монтирования ФС. Сначала указывается раздел блочного устройства, а потом "точка монтирования".
Если ввести mount без опций и аргументов, то мы увидим какие разделы сейчас смонтированы в системе.

```
└─# mount
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
udev on /dev type devtmpfs (rw,nosuid,relatime,size=942584k,nr_inodes=235646,mode=755,inode64)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=600,ptmxmode=000)
tmpfs on /run type tmpfs (rw,nosuid,nodev,noexec,relatime,size=202104k,mode=755,inode64)
/dev/sda1 on / type ext4 (rw,relatime,errors=remount-ro)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,inode64)
cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec,relatime,nsdelegate,memory_recursiveprot)
pstore on /sys/fs/pstore type pstore (rw,nosuid,nodev,noexec,relatime)
bpf on /sys/fs/bpf type bpf (rw,nosuid,nodev,noexec,relatime,mode=700)
systemd-1 on /proc/sys/fs/binfmt_misc type autofs (rw,relatime,fd=40,pgrp=1,timeout=0,minproto=5,maxproto=5,d
ect,pipe_ino=4856)
hugetlbfs on /dev/hugepages type hugetlbfs (rw,nosuid,nodev,relatime,pagesize=2M)
debugfs on /sys/kernel/debug type debugfs (rw,nosuid,nodev,noexec,relatime)
mqueue on /dev/mqueue type mqueue (rw,nosuid,nodev,noexec,relatime)
tmpfs on /run/lock type tmpfs (rw,nosuid,nodev,noexec,relatime,size=5120k,inode64)
tracefs on /sys/kernel/tracing type tracefs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /run/credentials/systemd-journald.service type tmpfs (ro,nosuid,nodev,noexec,relatime,nosymfollow,si
=1024k,nr_inodes=1024,mode=700,inode64,noswap)
configfs on /sys/kernel/config type configfs (rw,nosuid,nodev,noexec,relatime)
fusectl on /sys/fs/fuse/connections type fusectl (rw,nosuid,nodev,noexec,relatime)
tmpfs on /tmp type tmpfs (rw,nosuid,nodev,nr_inodes=1048576,inode64)
binfmt_misc on /proc/sys/fs/binfmt_misc type binfmt_misc (rw,nosuid,nodev,noexec,relatime)
sunrpc on /run/rpc_pipefs type rpc_pipefs (rw,relatime)
tmpfs on /run/credentials/getty@tty1.service type tmpfs (ro,nosuid,nodev,noexec,relatime,nosymfollow,size=102
,nr_inodes=1024,mode=700,inode64,noswap)
tmpfs on /run/user/1000 type tmpfs (rw,nosuid,nodev,relatime,size=202104k,nr_inodes=50526,mode=700,uid=1000,g
=1000,inode64)
gvfsd-fuse on /run/user/1000/gvfs type fuse.gvfsd-fuse (rw,nosuid,nodev,relatime,user_id=1000,group_id=1000)
portal on /run/user/1000/doc type fuse.portal (rw,nosuid,nodev,relatime,user_id=1000,group_id=1000)

┌──(root㉿kali)-[~]
└─#
```

Аналогично можно сделать, введя cat /proc/mounts.

```
└─# cat /proc/mounts
sysfs /sys sysfs rw,nosuid,nodev,noexec,relatime 0 0
proc /proc proc rw,nosuid,nodev,noexec,relatime 0 0
udev /dev devtmpfs rw,nosuid,relatime,size=942584k,nr_inodes=235646,mode=755,inode64 0 0
devpts /dev/pts devpts rw,nosuid,noexec,relatime,gid=5,mode=600,ptmxmode=000 0 0
tmpfs /run tmpfs rw,nosuid,nodev,noexec,relatime,size=202104k,mode=755,inode64 0 0
/dev/sda1 / ext4 rw,relatime,errors=remount-ro 0 0
securityfs /sys/kernel/security securityfs rw,nosuid,nodev,noexec,relatime 0 0
tmpfs /dev/shm tmpfs rw,nosuid,nodev,inode64 0 0
cgroup2 /sys/fs/cgroup cgroup2 rw,nosuid,nodev,noexec,relatime,nsdelegate,memory_recursiveprot 0 0
pstore /sys/fs/pstore pstore rw,nosuid,nodev,noexec,relatime 0 0
bpf /sys/fs/bpf bpf rw,nosuid,nodev,noexec,relatime,mode=700 0 0
systemd-1 /proc/sys/fs/binfmt_misc autofs rw,relatime,fd=40,pgrp=1,timeout=0,minproto=5,maxproto=5,direct,pipe_
ino=4856 0 0
hugetlbfs /dev/hugepages hugetlbfs rw,nosuid,nodev,relatime,pagesize=2M 0 0
debugfs /sys/kernel/debug debugfs rw,nosuid,nodev,noexec,relatime 0 0
mqueue /dev/mqueue mqueue rw,nosuid,nodev,noexec,relatime 0 0
tmpfs /run/lock tmpfs rw,nosuid,nodev,noexec,relatime,size=5120k,inode64 0 0
tracefs /sys/kernel/tracing tracefs rw,nosuid,nodev,noexec,relatime 0 0
tmpfs /run/credentials/systemd-journald.service tmpfs ro,nosuid,nodev,noexec,relatime,nosymfollow,size=1024k,nr
_inodes=1024,mode=700,inode64,noswap 0 0
configfs /sys/kernel/config configfs rw,nosuid,nodev,noexec,relatime 0 0
fusectl /sys/fs/fuse/connections fusectl rw,nosuid,nodev,noexec,relatime 0 0
tmpfs /tmp tmpfs rw,nosuid,nodev,nr_inodes=1048576,inode64 0 0
binfmt_misc /proc/sys/fs/binfmt_misc binfmt_misc rw,nosuid,nodev,noexec,relatime 0 0
sunrpc /run/rpc_pipefs rpc_pipefs rw,relatime 0 0
tmpfs /run/credentials/getty@tty1.service tmpfs ro,nosuid,nodev,noexec,relatime,nosymfollow,size=1024k,nr_inode
s=1024,mode=700,inode64,noswap 0 0
tmpfs /run/user/1000 tmpfs rw,nosuid,nodev,relatime,size=202104k,nr_inodes=50526,mode=700,uid=1000,gid=1000,ino
de64 0 0
gvfsd-fuse /run/user/1000/gvfs fuse.gvfsd-fuse rw,nosuid,nodev,relatime,user_id=1000,group_id=1000 0 0
portal /run/user/1000/doc fuse.portal rw,nosuid,nodev,relatime,user_id=1000,group_id=1000 0 0

┌──(root㉿kali)-[~]
└─# sS
```

6. fstab

Файл /etc/fstab сообщает ядру о доступных для монтирования файловых системах.

6.1 Образец fstab

Образец файла /etc/fstab выглядит примерно так:

```
<fs> <mountpoint> <type>    <opts>            <dump/pass>

/dev/hda1       /boot           ext3            noauto,noatime          1 1
/dev/hda3       /               reiserfs        noatime                 0 0
/dev/hda2       none            swap            sw                      0 0
/dev/cdrom      /mnt/cdrom      iso9660         noauto,ro,user          0 0
# /proc should always be enabled
proc            /proc           proc            defaults                0 0
```

Каждая некомментированная строка определяет раздел блочного устройства., точку монтирования, тип ФС и ее опции, используемые при монтировании, а также еще 2 числовых поля.
Числовые поля:

1. сообщение о том, какие файловые требуют резервного копирования при помощи команды dump
2. программная проверка целостности ФС fsck и установление порядка в соответствии с которым должны проверяться файловые системы во время загрузки

В строке /proc есть опция defaults. Ее следует использовать, если мы хотим смонтировать ФС со стандартными опциями.

При использовании опции user системе сообщается, что данная конкретная ФС может монтироваться любым пользователем. Это удобно для съемных носителей, например для CD-ROM. Без этой опции CD-ROM мог бы использоваться только рут.

6.2 Размонтирование файловых систем

Как правило при выключении или перезагрузке все файловые системы, которые были подключены, размонтируются автоматически.
Также можно размонтировать ФС и вручную. Для этого можно использовать команду umount. Например umount /mnt или umount /dev/hda3.
После размонтирования, любые файлы в /mnt, которые были сокрыты прежде подключенной ФС, станут доступны.

7. fsck

Если в системе произойдет сбой или она заблокируется, то система не будет иметь возможности корректно отмонтировать ФС. В таком случае наши ФС окажутся в непредсказуемом состоянии и их целостность может быть нарушена.
Когда система перезагрузится программа fsck определит, что ФС не были корректно размонтированы и попытается произвести проверку целостности ФС.

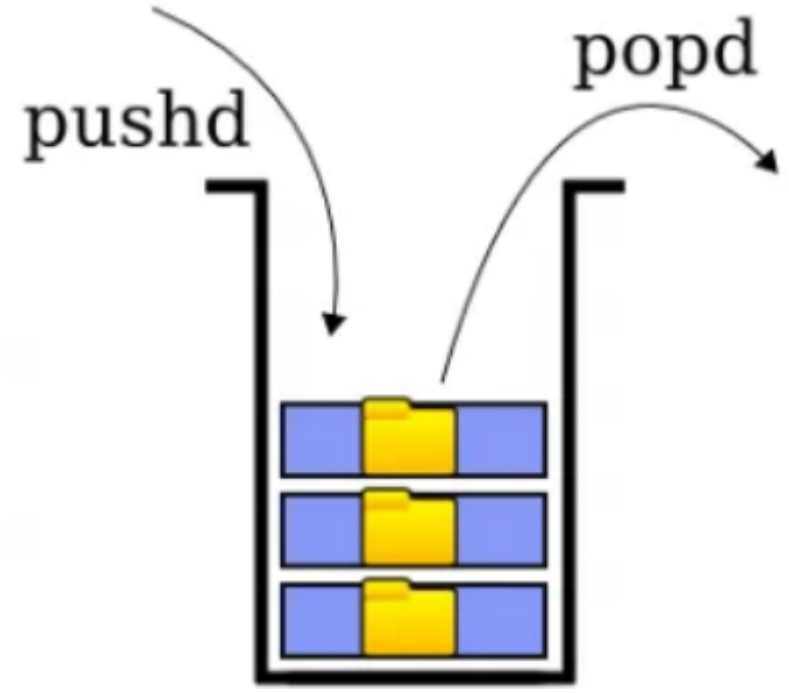
Для ФС, которая будет проверена с помощью fsck, должно быть не нулевое значение в поле “pass” (последнее поле) в /etc/fstab. Обычно, для корневой ФС значение устанавливается в “1”, указывая на то, что она должна быть проверена в первую очередь. У всех других ФС, которые должны быть проверены во время загрузки, значение поля “pass” должно быть “2” или выше. Для некоторых журналируемых ФС безопасно иметь значение “0”, поскольку сам программный код журналирования (а не внешний fsck) заботится о сохранении целостности ФС.

В случае, когда после перезагрузки fsck не может полностью восстановить поврежденную ФС, требуется перейти в однопользовательский режим и запустить fsck вручную, передавая в качестве аргумента блочное устройство конкретного раздела.

Проблема сканирования fsck в том, что оно может занять длительное время до завершения, тк совокупность метаданных файловой системы (внутренняя структура данных) должна быть просканирована., чтобы убедиться в их целостности.

8. Команды pushd и popd

pushd и popd - команды, позволяющие работать нам со стеком каталогов и изменять текущий рабочий каталог.



Выше видна схема работы команд pushd и popd

8.1 Стек каталогов

Стек каталогов - это список каталогов, к которым ранее были обращения. Содержимое стека каталогов можно увидеть при помощи ввода команды dirs.

```
┌──(root㉿kali)-[~]
└─# exit

┌──(kali㉿kali)-[~]
└─$ dirs
~

┌──(kali㉿kali)-[~]
└─$
```

Как видно, он сейчас пуст.
Каталоги добавляются в стек при переходе в каталог с помощью команды pushd, а удаляются с помощью popd.

Текущий рабочий каталог всегда находится на вершине стека каталогов. Текущий рабочий каталог является каталогом (папка), в котором в настоящее время работает пользователь. Каждый раз, когда вы взаимодействуете с командной строкой, вы работаете в директории.

При помощи команды pwd можно узнать в какой сейчас директории мы находимся.

```
┌──(kali㉿kali)-[~]
└─$ pwd
/home/kali

┌──(kali㉿kali)-[~]
└─$
```

Во время навигации по файловой системе можно использовать таб (Tab) для автозаполнения имен каталогов.

8.2 pushd

Пример: pushd /var/www в случае успеха команда напечатает стек каталогов.
pushd сначала сохраняет текущий рабочий каталог в верхнюю часть стека, а после переходит к указанному каталогу.
тк текущий каталог всегда должен быть на вершине стека, то после изменения новый текущий каталог попадает на вершину стека, но не сохраняется в стеке. Чтобы сохранить его надо вызвать pushd. Если использовать cd для перехода в другой каталог, то верхний элемент стека будет потерян.
Попробуем добавить несколько каталогов в стек.

```
┌──(kali㉿kali)-[~]
└─$ pwd
/home/kali

┌──(kali㉿kali)-[~]
└─$ pushd /var/www
/var/www ~

┌──(kali㉿kali)-[/var/www]
└─$ pushd /opt
/opt /var/www ~

┌──(kali㉿kali)-[/opt]
└─$
```

Для того, чтобы отменить изменение по умолчанию для каталога, то надо использовать опцию -n. В таком случае, например, чтобы добавить каталог /usr/local в стек, но при этом не переходить в него, а остаться в текущем, стоит ввести pushd -n /usr/local.

```
┌──(kali㉿kali)-[/opt]
└─$ pushd -n /usr/local
/opt /usr/local /var/www ~

┌──(kali㉿kali)-[/opt]
└─$
```

Команда pushd принимает варианты +N и -N для перемещения между директориями стека. Параметр +N изменяется на N-ный элемент списка стека слева направо, а -N справа налево.
Теперь проиллюстрируем параметры.

```
┌──(kali㉿kali)-[/opt]
└─$ dirs -l -v
 0  /opt
 1  /usr/local
 2  /var/www
 3  /home/kali

┌──(kali㉿kali)-[/opt]
└─$
```

Так, например для перехода в каталог /var/www и поместить его в верх стека, можно как написать pushd +2, так и pushd -1.
При использовании pushd без аргументов две верхние директории переключаются и делается новая вершина стека текущей. То есть по сути тоже, что и при использовании cd.

8.3 popd

При использовании popd без аргументов удаляется верхний каталог из стека и происходит переход в новый верхний каталог. Например:

```
┌──(kali㉿kali)-[/opt]
└─$ pushd -n /usr/local
/opt /usr/local /var/www ~

┌──(kali㉿kali)-[/opt]
└─$ dirs -l -v
 0  /opt
 1  /usr/local
 2  /var/www
 3  /home/kali

┌──(kali㉿kali)-[/opt]
└─$ popd
/usr/local /var/www ~

┌──(kali㉿kali)-[/usr/local]
└─$
```

Опция -n подавляет изменение каталога по умолчанию и удаляет второй элемент из стека.

```
┌──(kali㉿kali)-[/opt]
└─$ popd
/usr/local /var/www ~

┌──(kali㉿kali)-[/usr/local]
└─$ popd -n
/usr/local ~

┌──(kali㉿kali)-[/usr/local]
└─$
```

Аналогично pushd, popd может принимать +N и -N для удаления N-ного каталога из стека.